Garmin Coding Assignment

Alexander Alves

October 27th 2025

1 Submitted Files

This submission contains three files:

- **run_length_encoding.c**: Implements the compression and decompression functions.
- **compression_testing.c**: A test runner used to verify the correctness of the compression and decompression functions against relevant test cases.
- **Makefile**: Can build and run the test program. It can be built with `make`, executed with `make run`, and cleaned with `make clean`.

The code assumes a standard C toolchain.

2 Design Description

2.1 Implementation

The compressor performs in place run-length encoding using the following format:

- A leading `0xFF` flag byte indicating that the buffer has been compressed. This was chosen as the flag, since the values are limited between `0x00` and `0x7F`, so `0xFF` will never show up as a value.
- A sequence of (`count`, `value`) byte pairs, repeated n times.

This means that a successfully compressed buffer will have a total size of $2n + 1$ bytes, where n is the number of (`count`, `value`) pairs.

The compression function uses a two pass approach:

1. **Pass 1:** The input buffer is scanned to identify runs of repeated bytes. Each run is stored as a `Value_Info` struct containing a count and corresponding value. Runs longer than 255 are split into multiple pairs of (255, `value`) plus a final pair for the remaining count. The function then calculates the potential compressed size and compares it to the original buffer size.

2. **Pass 2:** If the compressed size is smaller, the buffer is overwritten in place with the `0xFF` flag byte followed by all (`count`, `value`) pairs. Otherwise, the function leaves the buffer and its size unchanged.

The decompression function does the opposite of the compression process, returning the buffer to its original form from the run length encoded format described above.

This function also uses a two pass approach:

1. **Pass 1:** The function checks whether the buffer begins with the compression flag byte `0xFF`. If the flag isnt there, the buffer is assumed to be uncompressed and is left unchanged. If present, the function scans the remaining bytes to calculate the expected decompressed size based on all (`count`, `value`) pairs.

2. **Pass 2:** If the calculated decompressed size fits within the current buffer capacity (provided as a function parameter), the function reconstructs the original data in place by expanding each pair into its repeated sequence of values. If the size exceeds the available space, or if an error occurs, the buffer remains unchanged and its original size is returned.

Both the compression and decompression functions run in linear time relative to the input. They also require linear extra space to temporarily allocate the (`count`, `value`) pairs on the heap during the first passes.

All buffer sizes are stored as `size_t` to ensure they are unsigned and can hold large values. Counts and values use `unsigned char` since values are positive and capped at `0x7F`, while counts have to be able to reach `0xFF` to maximize run length per pair.

2.2 Strengths and Weaknesses

This implementation works well for larger buffers, where long runs can take advantage of the count byte. It does not perform as well on smaller buffers with short runs, since any non sequential value adds an extra byte and can result in little to no size benefit. However, the solution does mitigate potential size loss from this by not modifying the buffer and returning the original size if there would be a net expansion.

Using a flag, as this implementation does with 0xFF at the start of any compressed array, has advantages. It allows the decompressor to immediately determine whether a buffer needs processing, which saves time and prevents the decompressor from operating on an invalid buffer. The trade off is that the flag adds an extra byte, slightly reducing compression efficiency, especially for small buffers.

2.3 Potential Improvements

Currently, the functions do not provide a return value that explicitly indicates whether compression or decompression occurred. Users must be able to tell this from the returned size or by the 0xFF flag as the first element in the buffer. This was done to follow the assignment's return specifications, but could be modified in future implementations for more clarity.

Another potential improvement involves the handling of count bytes. In the current implementation, counts can reach the maximum value of 255, which is efficient for large buffers but less useful for small buffers with mostly short runs. For small buffer applications with values limited to 0x00–0x7F, counts could instead be distinguished from values using the 0x80–0xFF range. This would allow counts to only represent runs of 3 or more, eliminating the need for a count byte for single or double occurrences and improving compression efficiency on small buffers. The trade-off would be reduced efficiency for very long runs (greater than 127 values in a sequence) and reduced adaptability if values outside 0x7F are required in the future.

3 Testing Strategy

The functions were tested using a test runner that contained a variety of predefined buffers to cover different use cases. These included long runs to verify compression efficiency, buffers with no repeats to check worst case behavior, boundary cases around the maximum count byte, small and empty buffers, and large buffers with mixed run lengths. For each buffer, the test runner compresses it, checks whether the output matches the expected compressed form, then decompresses it and verifies that the original data is correctly restored. The tests also checked that edge cases, like insufficient buffer capacity during decompression, are handled properly.

The code was compiled with `-Wall` and `-Wextra` and additionally checked with Valgrind during these tests to confirm that no warnings, errors, or memory leaks occurred.